int3ract: Johnson–Neyman Technique and Its Three-Way Extension for Frequentist and Bayesian Models in R

Robert W. Krause

Interaction effects are ubiquitous in applied statistical modelling, yet their meaningful interpretation remains challenging. The classic Johnson–Neyman (JN) technique (Johnson and Neyman 1936) addresses this challenge for two-way interactions by identifying the regions of a moderator's range over which a focal effect is and is not statistically significant. The **int3ract** package for R implements the JN technique and its three-way extension (the Johnson–Neyman–Krause, or JNK, technique) for both frequentist and Bayesian models. The function `JNK_freq()` auto-detects models fitted via `lm()`/`glm()`, **RSiena**'s `siena()`, or **lme4**'s `lmer()`/`glmer()`, but can also be applied to multiplicative interactions from (virtually) any model family by supplying a coefficient vector and covariance matrix directly. For Bayesian Stochastic Actor-Oriented Models (SAOMs) estimated with **multiSiena**, or any model producing posterior draws, the function `JNK_bayes()` produces conditional posterior distributions. For two-way interactions, classic shaded confidence-band plots are created that visually demarcate significant and non-significant regions along the moderator range; three-way interactions yield colour-gradient heatmaps with optional crosshatch overlays for non-significant regions. The package is designed to encourage richer, region-specific reporting of interaction effects in place of the conventional single-slope spotlight approach.

1 Introduction

Multiplicative interaction terms allow researchers to examine whether the relationship between a predictor X and an outcome Y depends on the value of one or more additional variables—the so-called moderators. Interpreting such interactions by inspecting the sign and significance of the interaction coefficient alone is, however, insufficient and potentially misleading (Bauer and Curran 2005; Preacher, Curran, and Bauer 2006). The key insight behind the Johnson–Neyman (JN) technique is that instead of evaluating the focal effect at a small number of

selected moderator values (often referred to as "spotlight" or "simple-slope"), one can derive the *entire* region of the moderator's distribution over which the focal effect is statistically distinguishable from zero (Johnson and Neyman 1936; Potthoff 1964).

Originally proposed in the context of the analysis of covariance (Johnson and Neyman 1936), the JN technique was later adapted for regression (Potthoff 1964) and updated by Bauer and Curran (2005), whose formulation is closest to what **int3ract** implements. Several R packages already address the two-way JN case, for instance, **interactions** (Long 2019) and **jtools** (Long 2020) but none does so for three-way interactions. The contribution of **int3ract** is threefold.

1. **Three-way interactions.** The package provides a complete implementation of the JNK technique for three-way multiplicative interactions, producing a heatmap over the two-dimensional moderator grid rather than a single-axis ribbon plot.
2. **Model-agnostic frequentist engine.** `JNK_freq()` auto-detects `lm`, `glm`, `sienaFit`, and `lmerMod`/`glmerMod` objects, but also accepts a raw coefficient vector and covariance sub-matrix. It therefore applies to any model that produces these quantities – ordinary least squares, generalised linear models, mixed-effects models, survival models, etc. – without modification. For **lme4** models with random interaction terms, per-group Johnson–Neyman plots are available via `fixed_only = FALSE`.
3. **Bayesian models.** The function `JNK_bayes()` adapts the technique to the posterior distribution rather than point estimates, producing density plots (two-way) and posterior-mean heatmaps with Bayesian p-value overlays (three-way). It accepts **multiSiena** objects directly or raw posterior-draw matrices from any Bayesian estimation.

The remainder of the paper is organised as follows. Section 2 reviews the statistical background of the JN technique and its three-way extension. Section 3 describes the package architecture. Section 4 and Section 5 illustrate the frequentist and Bayesian workflows with worked examples. Section 6 introduces the application to Stochastic Actor-Oriented Models (SAOMs) estimated with **RSiena** and **multiSiena**. Section 7 discusses graphical customisation. Section 8 concludes.

2 Statistical Background

2.1 The Two-Way Johnson–Neyman Technique

Consider the linear model

$$Y_i = \beta_0 + \beta_1 X_i + \beta_2 M_i + \beta_3 X_i M_i + \varepsilon_i, \tag{1}$$

where X is the focal predictor, M is the moderator, and β_3 is the interaction coefficient. The simple slope of Y with respect to X at a given value m of the moderator is

$$\hat{\theta}_X(m) = \hat{\beta}_1 + \hat{\beta}_3\, m. \tag{2}$$

Its sampling variance follows from the delta method:

$$\widehat{Var}[\hat{\theta}_X(m)] = \widehat{Var}(\hat{\beta}_1) + 2m\,\widehat{Cov}(\hat{\beta}_1, \hat{\beta}_3) + m^2\,\widehat{Var}(\hat{\beta}_3). \tag{3}$$

The two-way JN regions are the set of moderator values m for which the two-sided z-test

$$z(m) = \frac{\hat{\theta}_X(m)}{\sqrt{\widehat{Var}[\hat{\theta}_X(m)]}}$$

satisfies $|z(m)| > z_{\alpha/2}$, where $z_{\alpha/2}$ is the $(\alpha/2)$-upper quantile of the standard normal distribution.

The symmetry of Equation 1 implies that the roles of X and M are interchangeable: the simple slope of Y on M moderated by X is $\hat{\beta}_2 + \hat{\beta}_3\, x$, with an analogous variance formula. **int3ract** therefore always produces *two* JN plots per two-way interaction and three for three-way interactions – one for each variable acting as the focal predictor.

2.2 The Three-Way Extension

When a third variable W enters as an additional moderator, the full three-way interaction model is

$$\begin{aligned} Y_i = \beta_0 &+ \beta_1 X_i + \beta_2 M_i + \beta_3 W_i + \beta_4 X_i M_i + \beta_5 X_i W_i + \beta_6 M_i W_i \\ &+ \beta_7 X_i M_i W_i + \varepsilon_i. \end{aligned} \tag{4}$$

The conditional effect of X given M and W is

$$\theta_X(m, w) = \beta_1 + \beta_4 m + \beta_5 w + \beta_7 m w, \tag{5}$$

and the corresponding standard error is obtained via the multivariate delta method:

$$\begin{aligned} \widehat{Var}[\hat{\theta}_X(m, w)] = \widehat{Var}_{\beta_1} &+ m^2 \widehat{Var}_{\beta_4} + w^2 \widehat{Var}_{\beta_5} + (mw)^2 \widehat{Var}_{\beta_7} \\ &+ 2m\widehat{Cov}(\beta_1, \beta_4) + 2w\widehat{Cov}(\beta_1, \beta_5) + 2mw\widehat{Cov}(\beta_1, \beta_7) \\ &+ 2mw\widehat{Cov}(\beta_4, \beta_5) + 2m^2 w\widehat{Cov}(\beta_4, \beta_7) \\ &+ 2mw^2\widehat{Cov}(\beta_5, \beta_7). \end{aligned} \tag{6}$$

Because the significance region is now two-dimensional, it cannot be depicted as a simple interval on a line. **int3ract** instead renders a heatmap of conditional point estimates $\hat{\theta}_X(m, w)$ using a diverging colour scale, and superimposes a crosshatch pattern over cells where the null hypothesis $\theta_X = 0$ should be rejected given the chosen α-threshold (default: `threshold = 0.05`).

3 Package Architecture

The package is organised around two exported functions and a shared utility layer, summarised in Table 1.

Table 1: Function hierarchy of **int3ract**. Both exported functions auto-detect the model class and extract the relevant quantities before delegating to internal engines in `utils.R`.

Function	Accepted input	Internal engine
`JNK_freq()`	`lm`, `glm`, `sienaFit`, `lmerMod`/`glmerMod`, or raw `covar`/`coefs`	`.freq_core()`, `parFun()`, `seFun()`
`JNK_bayes()`	`multiSiena`/`sienaBayesFit`, or raw posterior-draw matrix	`jnb_support2()`, `jnb_support3()`

Utility layer (`utils.R`). All internal helpers reside in a single file. `parFun()` and `seFun()` implement Equation 5 and Equation 6. Model-specific extractors (`.extract_lm()`, `.extract_siena()`, `.extract_lme4()`) handle the class-specific logic for `JNK_freq()`. The Bayesian support functions `jnb_support2()` and `jnb_support3()` produce the posterior-based plots for `JNK_bayes()`.

Frequentist engine (`JNK_freq()`). `JNK_freq()` auto-detects the class of its first argument and extracts coefficients, a covariance sub-matrix, variable names, and moderator value ranges. For `lm`/`glm` and `lmerMod`/`glmerMod` objects, interaction-term names are resolved automatically regardless of colon ordering. Researchers who can extract a coefficient vector and covariance matrix from any other model (whether fitted by `coxph()`, `betareg()`, or any other estimator) can call `JNK_freq()` directly by supplying `covar` and `coefs`.

For **lme4** models with random interaction terms, setting `fixed_only = FALSE` produces per-group Johnson–Neyman plots using the group-specific coefficients (fixed effects + BLUPs), mirroring the group-level output of `JNK_bayes()`.

Bayesian engine (`JNK_bayes()`). For Bayesian models there is no single-point estimate or analytic covariance matrix. Instead, conditional posteriors are used. `JNK_bayes()` accepts either a raw matrix of posterior draws or a `multiSiena`/`sienaBayesFit` object (handling burn-in removal, thinning, and detection of fixed vs. random parameters).

For two-way interactions, `JNK_bayes()` computes the conditional posterior draws

$$\tilde{\theta}_X^{(s)}(m) = \theta_X^{(s)} + \theta_{XM}^{(s)} \cdot m$$

at each sampled iteration s and moderator value m, yielding a full conditional posterior distribution. These are visualised as overlapping density curves coloured by moderator value, and summarised by the posterior mean, posterior standard deviation, and Bayesian p-value $\Pr(\tilde{\theta}_X > 0 \mid \text{data}, \text{model}, \text{prior})$.

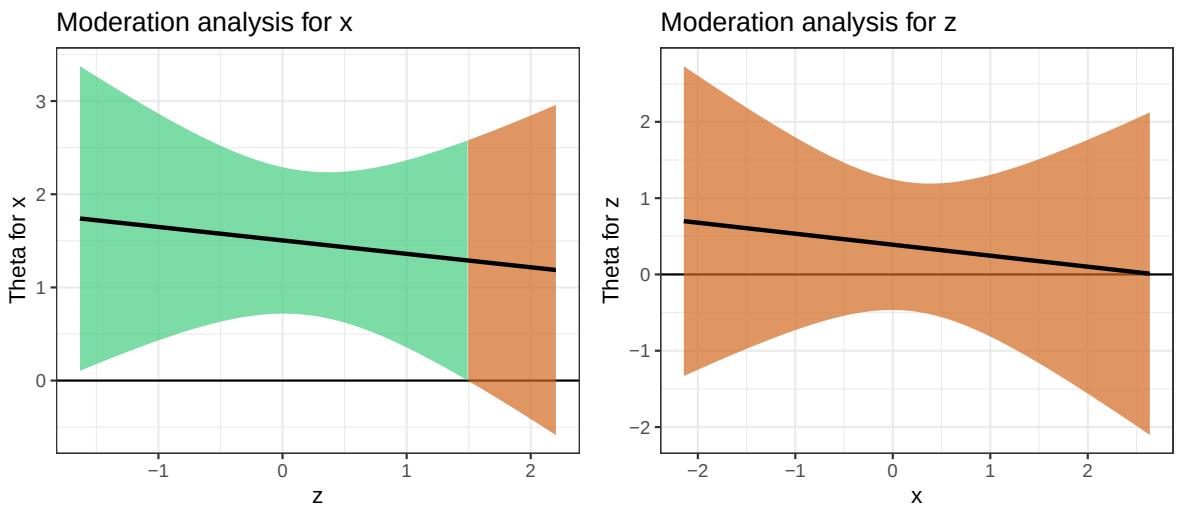
For three-way interactions, this is extended to the grid (m, w), producing heatmaps of both the posterior mean and the Bayesian p-value, each overlaid with a crosshatch on cells outside the user-specified "significance" thresholds.

4 Frequentist Usage

4.1 Two-Way Interaction with `lm`

The simplest entry point is `JNK_freq()` applied to an ordinary least-squares model.

```
library(int3ract)

set.seed(1402)
dat <- data.frame(x = rnorm(100),
                  z = rnorm(100),
                  w = rnorm(100))

dat$y = dat$x + 0.5 * dat$z + -.5 * dat$w + 0.5 * dat$x *
  dat$z * dat$w + rnorm(100,sd = 4)

mod2 <- lm(y ~ x * z, data = dat)

jn2 <- JNK_freq(mod2,
                theta_1 = "x",
                theta_2 = "z",
                alpha   = 0.05)
```



(a) Conditional effect of `x` as a function of the moderator `z`.

(b) Conditional effect of `z` as a function of the moderator `x`.

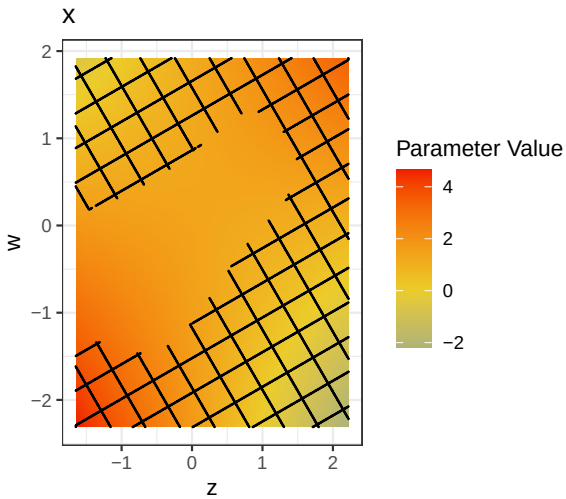
Figure 1: Johnson–Neyman plots for the two-way interaction between `x` and `z` ($\alpha = 0.05$). The ribbon shows pointwise 95% confidence bands for the conditional effect; green shading indicates regions where the effect is statistically significant, chocolate where it is not.

The returned list `jn2` contains `$param_table`, a list of two data frames (one per each variable involved in the interactions) with columns for the moderator values (`mod_vals`), conditional parameter, its standard error, and p-value (`theta_vals`, `theta_se`, and `theta_p`), and a logical whether it is significant given the chosen `threshold`; and `$plots`, a list of two **ggplot2** objects.

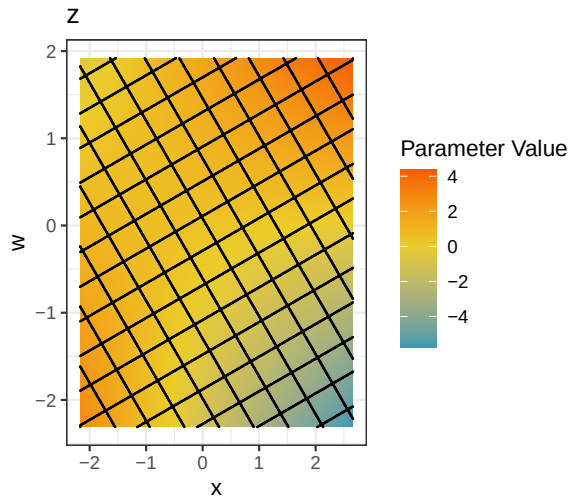
In each plot the ribbon is filled with `sig_color` (default `'seagreen3'`) where $|z| > z_{\alpha/2}$ and with `non_sig_color` (default `'chocolate'`) elsewhere.

4.2 Three-Way Interaction with `lm`

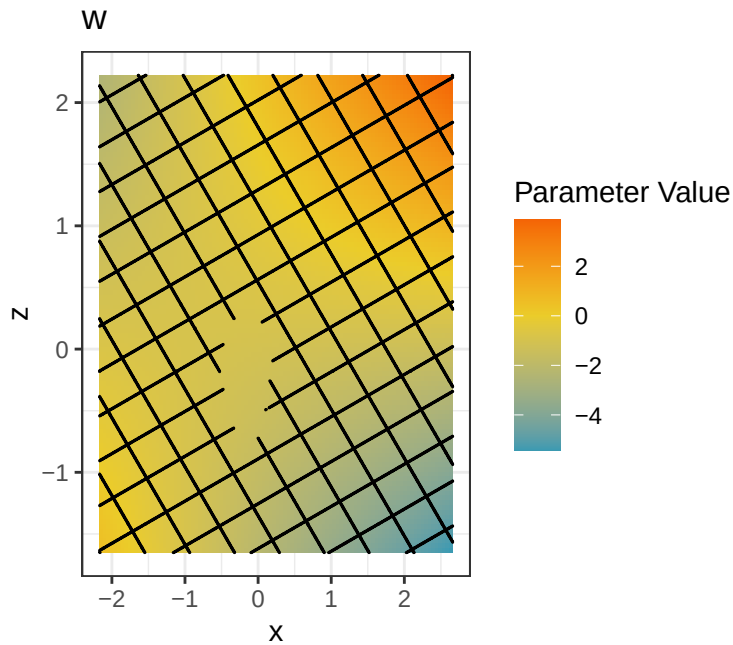
```
mod3 <- lm(y ~ x * z * w, data = dat)

jn3 <- JNK_freq(mod3,
                theta_1    = "x",
                theta_2    = "z",
                theta_3    = "w",
                range_size = 100)
```



(a) Conditional effect of `x` across the (`z`, `w`) moderator grid.



(b) Conditional effect of `z` across the (`x`, `w`) moderator grid.



(c) Conditional effect of `w` across the (`x`, `z`) moderator grid.

Figure 2: Johnson–Neyman–Krause heatmaps for the three-way interaction among `x`, `z`, and `w` (`range_size = 50`, $\alpha = 0.05$). Each cell shows the conditional point estimate of the focal effect across the two-dimensional moderator grid; the diverging colour scale runs from blue (negative) through yellow (zero) to red (positive). Crosshatched cells indicate regions where the conditional effect is not statistically significant.

Setting `range_size = 50` produces a 50×50 grid for each of the three heatmaps (i.e., $3 \times 2{,}500$ conditional effect evaluations). When all three moderator ranges contain fewer than eight distinct values, the point estimate is printed in each cell; otherwise only the colour gradient communicates the magnitude. The ranges for each variable are by default extracted from the `lm`/`glm`/`lmerMod` object but can also be supplied directly, which is always necessary for `sienaFit` objects and the generic `covar`/`coefs` path (see the code chunks below).

4.3 Mixed-Effects Models (lme4)

`JNK_freq()` natively supports `lmerMod` and `glmerMod` objects from **lme4**. For fixed effects the interface is identical to `lm`:

```
library(lme4)
library(int3ract)

lme_mod <- lmer(y ~ x * z + (x * z | group), data = dat)

jn_lme <- JNK_freq(lme_mod,
                   theta_1 = "x",
                   theta_2 = "z")
```

When the interaction terms are random, setting `fixed_only = FALSE` produces per-group Johnson–Neyman plots using group-specific coefficients (fixed effects + BLUPs). The grouping variable defaults to the first random-effects term but can be specified via `group_var`:

```
jn_groups <- JNK_freq(lme_mod,
                      theta_1    = "x",
                      theta_2    = "z",
                      fixed_only = FALSE,
                      group_var  = "group")
```

4.4 Direct Use with Coefficient Vectors

Because `JNK_freq()` also accepts a raw coefficient vector and covariance matrix, it is straightforward to apply it to models not covered by the built-in extractors or to Bayesian models when the posterior is reduced to a covariance matrix and point estimates:

```
JNK_freq(covar = my_vcov[c("x","z","x:z"), c("x","z","x:z")],
         coefs = my_coefs[c("x","z","x:z")],
         name  = c("x", "z"),
```

```
         theta_1 = "x", theta_2 = "z",
         theta_1_vals = c(-3, 3),
         theta_2_vals = c(-3, 3))
```

The same pattern applies to survival models (**survival**), ordinal regression (**MASS**), beta regression (**betareg**), and any other estimation that provides parameter estimates and their covariances.

5 Bayesian Usage

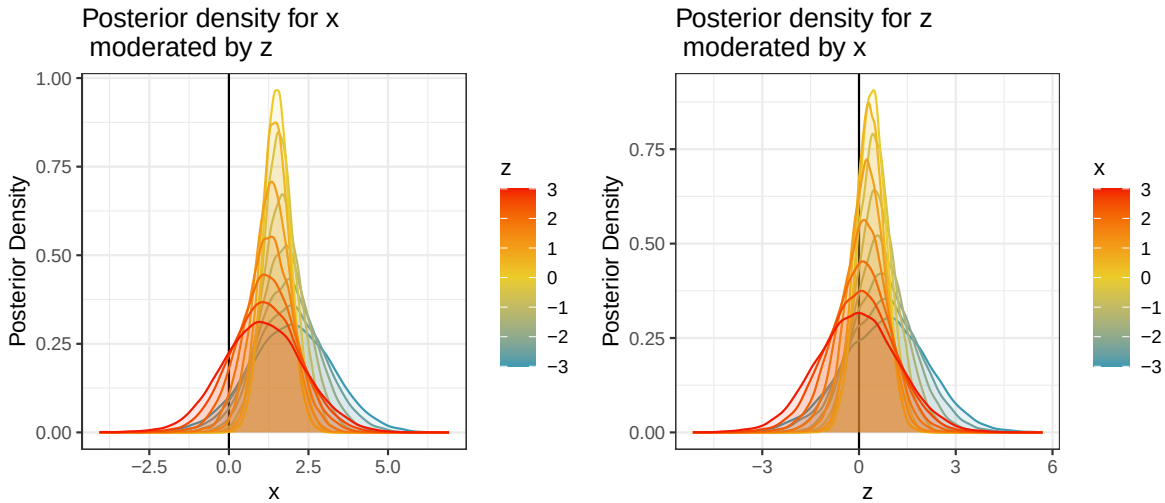
Using `JNK_bayes()` is as straightforward as using `JNK_freq()`. The function either expects a posterior matrix or sienaBayesFit/multiSiena object, from which it extracts the relevant posterior draws. The same arguments for specifying the parameters and their values apply as for `JNK_freq()`, but instead of confidence intervals and p-values, the output includes conditional posterior means, standard deviations, and Bayesian p-values ($\Pr(\tilde{\theta}_X > 0 \mid \text{data}, \text{model}, \text{prior})$).

Parameters can either be specified by name (here, the posterior matrix `x` should be named accordingly) or by integer index (the position of the parameter in the posterior matrix). If names are given, only `theta_1`, `theta_2`, and, if applicable, `theta_3` need to be specified and the interactions are detected automatically (interactions should be named with the standard R notation for interactions, i.e., `x:z`).

The `thresholds` argument (a vector of length 2, e.g., `c(0.05, 0.95)`) determines which cells of the posterior-mean heatmap are crosshatched: cells whose Bayesian p-value falls inside the interval $[\text{threshold}_1, \text{threshold}_2]$ can be treated as 'non-significant'.

The function also handled burn-in iterations and thinning if desired via the the `thin` and `burn_in` arguments (default no thinning and no burn-in iterations).

```
library(MCMCpack)

mod_bayes2 <- MCMCregress(y ~ x * z, data = dat,
                          burnin = 1000, mcmc = 10000, thin = 1,
                          verbose = 0)


jnk_bayes2 <- JNK_bayes(mod_bayes2, theta_1 = 'x', theta_2 = 'z',
                        theta_1_vals = seq(-3,3,0.5),
                        theta_2_vals = seq(-3,3,0.5))
```



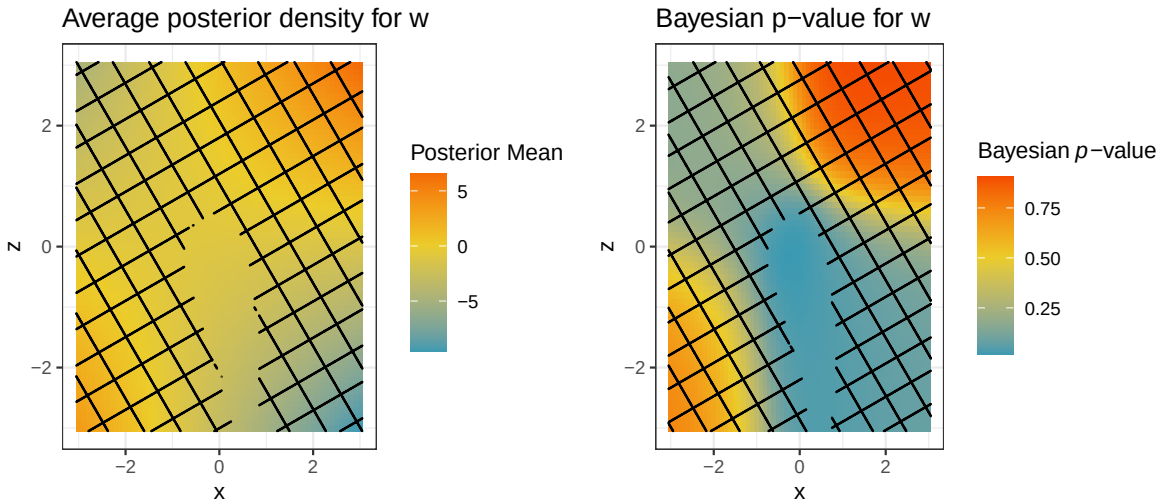
(a) Conditional effect of `x` as a function of the moderator `z`.

(b) Conditional effect of `z` as a function of the moderator `x`.

Figure 3: Johnson–Neyman–Krause plots for the two-way interaction between `x` and `z`. Each curve represents a conditional posterior distribution of the focal effect at a given value of the moderator.

Three-way plots in turn return heatmaps of the posterior mean and Bayesian p-value.

```
mod_bayes3 <- MCMCregress(y ~ x * z * w, data = dat,
                          burnin = 1000, mcmc = 10000, thin = 1,
                          verbose = 0)


jnk_bayes3 <- JNK_bayes(mod_bayes3,
                        theta_1 = 'x',
                        theta_2 = 'z',
                        theta_3 = 'w',
                        theta_1_vals = seq(-3,3,0.1),
                        theta_2_vals = seq(-3,3,0.1),
                        theta_3_vals = seq(-3,3,0.1),
                        thresholds = c(0.05, 0.95))
```



(a) Conditional posterior means of `w` as a function of the moderators `x` and `z`.

(b) Conditional posterior means of `z` as a function of the moderator `x`.

Figure 4: Johnson–Neyman–Krause plots for the three-way interaction between `x`, `z`, and `w`. The color gradient of the heatmap shows the value of conditional posterior mean of the focal effect. Crosshatch overlays indicate cells where the Bayesian p-value falls outside the threshold (here $p < 0.05$ or $p > 0.95$).

6 Stochastic Actor-Oriented Models

Interactions in SAOMs, estimated by **RSiena**'s `siena()` (formerly `siena07()`), between network statistics and actor attributes are common yet difficult to interpret (T. A. B. Snijders et al. 2026). Introducing SAOMs is out of the scope for this paper. If you wish to learn more about them, see for instance T. A. Snijders (2017). In short, SAOMs are models for network evolution, designed to identify the structural and covariate drivers that lead to the creation, dissolution, and maintenance of connections in networks.

The same `JNK_freq()` function handles `sienaFit` objects, using integer indices referencing the position of each effect in the **RSiena** effects object[1]. In `JNK_bayes()` the rate functions are skipped in the indexing! The moderator value ranges (`theta_1_vals`, `theta_2_vals`, `theta_3_vals`) should reflect the empirical support of the corresponding change statistics.

`JNK_bayes()` automatically detects whether each required parameter has been estimated as a fixed effect (η; shared across groups) or as a random effect (μ; group-varying, with a hyper-parameter), and extracts the appropriate posterior draws. When `hyper_only = FALSE` and

[1]While each effect in **RSiena** is named, within a given model, effects are not necessarily identifiable by their name but the same name might appear multiple times for different dependent variables or effect types, making secure unique indexing by integer much easier for users.

Table 2: Colour and pattern parameters available in all **int3ract** plotting functions. Default colour values are highlighted in their respective colour.

Parameter	Description	Default
sig_color	Ribbon fill for significant regions (2-way)	'seagreen3'
non_sig_color	Ribbon fill for non-significant regions (2-way)	'chocolate'
line_color	Point-estimate line colour (2-way)	'black'
color_low	Low end of diverging fill scale (heatmaps)	'#F05039'
color_mid	Midpoint colour of diverging fill scale	'#EBCC2A'
color_high	High end of diverging fill scale	'#000066'
color_grid	Crosshatch line colour	'black'
grid_density	Density of crosshatch lines	0.01
grid_spacing	Spacing between crosshatch lines	0.1
color_values	Within-cell text colour (heatmaps, 7 values)	'grey40'
crosshatch_non_sig	If TRUE, crosshatch marks the non-significant region	TRUE

at least one of the involved parameters is randomly varying between groups, `JNK_bayes()` additionally loops over all G groups and produces group-specific plots and tables, returned under the key `random_groups_effects`. This is useful for assessing heterogeneity of interaction effects across networks. It is strongly recommended to use `save = TRUE` when running `hyper_only = FALSE` so that all plots are directly created and saved. While this extends run-time, it will make it much easier to inspect the figures.

7 Graphical Output and Customisation

All plots are standard **ggplot2** objects (Wickham 2016) and can be further modified using **ggplot2**'s layer system. The crosshatch pattern for non-significant regions is produced by **ggpattern** (Stewart 2022). Table 2 documents the colour and pattern parameters shared across all plotting functions. The colours for high, mid, and low point for continuous plots are taken from the `Zissou1` palette from the **wesanderson** package (Ram and Wickham 2023). Direct use of palettes is not supported but users can always overlay their own color palettes post-hoc by calling `scale_fill_gradient2()`/`scale_color_gradient2()` (for more and alternatives see `?ggplot2::scale_fill_gradient()`).

8 Summary and Outlook

int3ract provides a unified interface for the Johnson–Neyman technique and its three-way extension across a broad class of frequentist and Bayesian models. The two exported

functions—`JNK_freq()` for frequentist models and `JNK_bayes()` for Bayesian models—auto-detect the input class and handle extraction internally, while remaining fully usable with raw coefficient vectors, covariance matrices, or posterior-draw matrices for models not covered by the built-in extractors. Native support for **lme4** mixed-effects models includes per-group Johnson–Neyman plots when interaction terms are random. Future development plans include:

1. A consideration for FDR-correction.
2. Additional Bayesian model classes beyond **multiSiena**.

Researchers are encouraged to report not only the JN boundaries but also the full plots as supplementary material, in keeping with calls for more transparent and complete reporting of interaction effects[2] (Simonsohn, Simmons, and Nelson 2020).

References

Allaire, JJ, and Christophe Dervieux. 2025. *Quarto: R Interface to 'Quarto' Markdown Publishing System*. https://doi.org/10.32614/CRAN.package.quarto.

Bates, Douglas, Martin Mächler, Ben Bolker, and Steve Walker. 2015. "Fitting Linear Mixed-Effects Models Using lme4." *Journal of Statistical Software* 67 (1): 1–48. https://doi.org/10.18637/jss.v067.i01.

Bauer, Daniel J., and Patrick J. Curran. 2005. "Probing Interactions in Fixed and Multilevel Regression: Inferential and Graphical Techniques." *Multivariate Behavioral Research* 40 (3): 373–400. https://doi.org/10.1207/s15327906mbr4003_5.

Iannone, Richard, Joe Cheng, Barret Schloerke, Shannon Haughton, Ellis Hughes, Alexandra Lauer, Romain François, JooYoung Seo, Ken Brevoort, and Olivier Roy. 2026. *Gt: Easily Create Presentation-Ready Display Tables*. https://doi.org/10.32614/CRAN.package.gt.

Johnson, Palmer O., and Jerzy Neyman. 1936. "Tests of Certain Linear Hypotheses and Their Applications to Some Educational Problems." *Statistical Research Memoirs* 1: 57–93.

Long, Jacob Scott. 2019. *interactions: Comprehensive, User-Friendly Toolkit for Probing Interactions*. https://cran.r-project.org/package=interactions.

———. 2020. *jtools: Analysis and Presentation of Social Scientific Data*. https://cran.r-project.org/package=jtools.

Müller, Kirill, and Hadley Wickham. 2026. *Tibble: Simple Data Frames*. https://doi.org/10.32614/CRAN.package.tibble.

Potthoff, Richard F. 1964. "On the Johnson–Neyman Technique and Some Extensions Thereof." *Psychometrika* 29 (3): 241–56. https://doi.org/10.1007/BF02289721.

[2] To give due credit, next to the already listed packages, **int3ract** also depends on **scales**, **tidyr**, and **tibble** (Wickham, Pedersen, and Seidel 2025; Wickham, Vaughan, and Girlich 2025; Müller and Wickham 2026). This paper was written using **gt**, **lme4**, **MCMCpack**, and of coruse **quarto** using **tinytex** (Iannone et al. 2026; Bates et al. 2015; Allaire and Dervieux 2025)

Preacher, Kristopher J., Patrick J. Curran, and Daniel J. Bauer. 2006. "Computational Tools for Probing Interactions in Multiple Linear Regression, Multilevel Modeling, and Latent Curve Analysis." *Journal of Educational and Behavioral Statistics* 31 (4): 437–48. https://doi.org/10.3102/10769986031004437.

Ram, Karthik, and Hadley Wickham. 2023. *Wesanderson: A Wes Anderson Palette Generator*. https://doi.org/10.32614/CRAN.package.wesanderson.

Simonsohn, Uri, Joseph P. Simmons, and Leif D. Nelson. 2020. "Specification Curve Analysis." *Nature Human Behaviour* 4 (11): 1208–14. https://doi.org/10.1038/s41562-020-0912-z.

Snijders, Tom A. B., Ruth M. Ripley, Krists Boitmanis, Christian Steglich, Nynke M. D. Niezink, Viviana Amati, and Felix Schoenenberger. 2026. *Siena - Simulation Investigation for Empirical Network Analysis*. Groningen, The Netherlands: University of Groningen. https://www.stats.ox.ac.uk/~snijders/siena/.

Snijders, Tom AB. 2017. "Stochastic Actor-Oriented Models for Network Dynamics." *Annual Review of Statistics and Its Application* 4 (1): 343–63.

Stewart, Mike. 2022. *ggpattern: ggplot2 Pattern Geoms*. https://cran.r-project.org/package=ggpattern.

Wickham, Hadley. 2016. *ggplot2: Elegant Graphics for Data Analysis*. New York: Springer-Verlag. https://doi.org/10.1007/978-3-319-24277-4.

Wickham, Hadley, Thomas Lin Pedersen, and Dana Seidel. 2025. *Scales: Scale Functions for Visualization*. https://doi.org/10.32614/CRAN.package.scales.

Wickham, Hadley, Davis Vaughan, and Maximilian Girlich. 2025. *Tidyr: Tidy Messy Data*. https://doi.org/10.32614/CRAN.package.tidyr.